Riwaz Shrestha
DS210
Collaborators: Ryden Tamura, Denali Schlesinger

HW8 Write Up

I used the provided template and added some extra traits to help me. I added Clone to the struct "DataFrame". Since it wasn't required, I didn't touch the MyError function but did use it to return errors.

My DataFrame struct is a hashmap containing titles as keys and vectors of the ColumnVal enum. I store a vector of titles in the struct so that I can maintain the order of the columns. I also keep a value for the number of rows in the data frame. I decided to keep track of the title types to make comparison between two data frames later on easier.

I didn't add much to the read_csv function since it was pretty much already set up. I mapped the first row to the titles vector in my dataframe and the given title types to my title types variable. The iteration through the rows was already set up, so all I had to do was insert the keys and values. I set the key to the title and used the "or_insert_with" operation to create a new vector and push the values. I also increased the row count that's stored in the DataFrame struct to keep everything updated.

```
                self.titles = r.iter().map(|s| s.to_string()).collect();
```

```
            // Put the data into the dataframe
            for (title, value) in self.titles.iter().zip(row.iter()) {

self.df.entry(title.clone()).or_insert_with(Vec::new).push(value.clone());
            }
            self.num_rows += 1;
            self.title_types = types.clone();
```

For my print statement, I iterate over the number of rows, print the row of titles. Then, for each row, I iterate through the titles and use an if let to access the vector stored in each title/key. I then get the ith value in the column and match it to the different ColumnVal options. I do this so that I can print the value itself and not the wrapper as well. Someone on Piazza mentioned the ":<#" operation, so shout out to them. I played around with the values and eventually created something I liked.

```
    fn print(&self) {
        for title in &self.titles{
            print!("{:<15}", title);
        }
        println!();
```

```
        for i in 0..self.num_rows as usize {
            for title in &self.titles {
                if let Some(column) = self.df.get(title) {
                    match &column[i] {
                        ColumnVal::One(s) => print!("{:<15}", s),
                        ColumnVal::Two(b) => print!("{:<15}", b),
                        ColumnVal::Three(f) => print!("{:<15}", f),
                        ColumnVal::Four(i) => print!("{:<15}", i),
                    }
                } else {
                    print!("None ");
                }
            }
            println!();
        }
    }
```

For the add_column function, much was already given. With the basics layed out, all I had to do was check if the column being added was of the same length or not. If it was, I just had to push the title into my titles vector and create a new entry in my hashmap using the title and the provided vector of ColumnVal.

```
    fn add_column(&mut self, name:String, col: Vec<ColumnVal>) -> Result<DataFrame,
MyError> {
        if col.len() == self.num_rows as usize {
            self.titles.push(name.clone());
            self.df.insert(name, col);
            Ok(self.clone())
        } else {
            Err(MyError(format!("Columns aren't of the same length!!")))
        }
    }
```

Merge_frame wasn't too hard to create. I checked if the types matched up by comparing the values in my titles vector and my title types vector. Storing this value made it so I wouldn't have to check each column's types. All I did for this function was iterate over the titles of self and extend the column with the values in the other data frame. I also increased the number of rows to keep it updated for the print statement.

```
    fn merge_frame(&mut self, df2: DataFrame) -> Result<DataFrame, MyError> {
        if self.titles == df2.titles && self.title_types == df2.title_types {
            for title in self.titles.iter() {
```

```
self.df.get_mut(title).unwrap().extend(df2.df.get(title).unwrap().iter().cloned());
            }
            self.num_rows += df2.num_rows;
            Ok(self.clone())
        } else {
            Err(MyError(format!("Columns or column types don't match!!")))
        }
    }
```

For the restrict_columns function, I first checked if the given columns were in my data frame or not by using .contains(). I then create a new DataFrame struct to store the filtered columns. I compare titles so that I can get the corresponding number of the title type. The number of rows doesn't change adn the types don't either. I then insert the vector of ColumnValues corresponding to the given filters and return the new DataFrame struct.

```
    fn restrict_columns(&mut self, cols: Vec<String>) -> Result<DataFrame, MyError> {
        for i in &cols {
            if self.titles.contains(&i) {
                continue;
            } else {
                return Err(MyError(format!("Column does not exist '{}'!!", i)));
            }
        }
        let mut restricted_df: DataFrame = DataFrame::new();
        let mut types: Vec<u32> = Vec::new();
        for i in &cols{
            for j in 0..self.titles.len(){
                if *i == self.titles[j]{
                    types.push(self.title_types[j])
                }
            }
        }
        restricted_df.num_rows = self.num_rows;
        restricted_df.title_types = types;
        restricted_df.titles.extend(cols.clone());
        for k in cols{
            restricted_df.df.insert(k.clone().to_string(),
self.df.get(&k).unwrap().clone());
        }
        Ok(restricted_df)
    }
```

For my filter function, I check if the column exists or not using .contains(). If it exists, I create a clone to do all my operations on so that I don't affect the original data frame. All I do in this function is iterate over the number of rows (in reverse to account for the possible indexing errors) and remove all rows that do not pass the closure. To remove the rows that don't return true, I iterate over the titles and remove all values in each vector of ColumnValues that correspond to the index of the row.

```
   fn filter(&mut self, label: &str, operation: fn(&ColumnVal) -> bool) ->
Result<DataFrame, MyError> {
       if !self.titles.contains(&label.to_string()){
           return Err(MyError(format!("Column does not exist '{}'!!", label)));
       }
       let mut clone: DataFrame = self.clone();
       let column = self.df.get(label).unwrap();
       for i in (0..(self.num_rows)).rev(){
           let val = &column[i as usize];
           if operation(val) {
               continue;
           } else {
               clone.num_rows -= 1;
               for j in &clone.titles{
                   clone.df.get_mut(j).unwrap().remove(i as usize);
               }
           }
       }
       Ok(clone)
   }
```

For column_op, I first create a vector of vectors corresponding to the titles given by iterating over the labels and appending the corresponding ones to the vector. I then apply the operation to the vector of vectors containing ColumnValues. `fn column_op(&mut self, labels: &[String], op: fn(&[Vec<ColumnVal>]) -> Vec<ColumnVal>) -> Vec<ColumnVal> {`

```
       let mut cols: Vec<Vec<ColumnVal>> = Vec::new();
       for label in labels {
           cols.push(self.df.get(label).unwrap().clone());
       }
       op(&cols)
   }
```

For the median function, I create a vector of labels to pass to column_op, create the median closure, then pass them into column_op. For the closure, I first get the wrapped value using a match statement, I store that in a vector of floats. I sort the floats using partial comparison and then find the median by dividing the length of the vector by 2. I get the average of the two

middle if the vector contains an even number of elements. Since the column_op returns a vector of ColumnValue, I had to use a match statement to access the float stored inside.

For sub_column, I pretty much did the same thing as for the median function, I pass the vector of labels and the closure. The closure makes sure that there are only two columns passed, iterates over the columns, and then subtracts them, putting the result into another vector of ColumnValues of type float.

In the main section, I called all the functions and labeled them.

Learning to use match statements was vital for this assignment. The closures were also far harder than I imagined they would be. I tried to use the lecture notes as much as possible but had to resort to ChatGPT to create the closures.

I didn't use ChatGPT for the majority of this HW. I used it to see how I would format the error message and to create the closures. I did use it as a search engine as well, but this was just to ask the basic questions like "what is the command to add a value to a vector" or "what is the command to add to a hashmap".

Fun fact: I spent about an hour reworking the read_csv function and couldn't figure out why it wasn't working even after comparing it to Ryden's code. The issue was that I am running the code locally and don't have auto-save on. Once I created the CSV files, I never saved them, so I spent a full hour trying to figure it out until I accidentally pressed cmd + s on the CSV file and it magically worked.

Output:

```
Print Data Frame:

Name          Number        PPG           YearBorn      TotalPoints   LikesPizza
Kareem        33            24.6          1947          48387         true
Karl          32            25.1          1963          46928         false
LeBron        23            27            1984          46381         false
Kobe          24            25            1978          43643         true
Michael       23            30.1          1963          42292         false

Print Data Frame 2:

Name          Number        PPG           YearBorn      TotalPoints   LikesPizza
Magic         32            19.6          1959          17707         true
Larry         33            24.3          1956          21791         true

Print Merged Data Frame:

Name          Number        PPG           YearBorn      TotalPoints   LikesPizza
Kareem        33            24.6          1947          48387         true
Karl          32            25.1          1963          46928         false
LeBron        23            27            1984          46381         false
Kobe          24            25            1978          43643         true
Michael       23            30.1          1963          42292         false
Magic         32            19.6          1959          17707         true
Larry         33            24.3          1956          21791         true

Add Hall of Fame Column:

Name          Number        PPG           YearBorn      TotalPoints   LikesPizza    Hall of Fame
Kareem        33            24.6          1947          48387         true          false
Karl          32            25.1          1963          46928         false         false
LeBron        23            27            1984          46381         false         true
Kobe          24            25            1978          43643         true          false
Michael       23            30.1          1963          42292         false         false
Magic         32            19.6          1959          17707         true          false
Larry         33            24.3          1956          21791         true          false

Restricted Data Frame (by name and total points):

Name          TotalPoints
Kareem        48387
Karl          46928
LeBron        46381
Kobe          43643
Michael       42292
Magic         17707
Larry         21791

Filetered Data Frame (over 25 ppg):

Name          Number        PPG           YearBorn      TotalPoints   LikesPizza    Hall of Fame
Karl          32            25.1          1963          46928         false         false
LeBron        23            27            1984          46381         false         true
Michael       23            30.1          1963          42292         false         false

Median of PPG: 25

TotalPoints - BirthYear:

46440
44965
44397
41665
40329
15748
19835
(base) riwazshrestha@crc-dot1x-nat-10-239-100-59 HW_8 %
```